

vLLM Ascend 赛道项目书

一、团队信息

项目名称	vLLM Ascend Lmcache 后端以及 encoder-only 适配
团队名称	Lone-wolf
队长姓名	fcqiao
队长联系方式	邮箱: qiao_sh_pudong@163.com 手机: 17701788526

二、环境依赖

组件	版本
vllm-ascned	v0.18.0rc1
vllm	v0.18.0
Python	>= 3.10
CANN	8.5.1
PyTorch	2.9.0
Triton Ascend	3.2.0
LMCache	0.3.12

三、参数理解与社区实现状态

1、--kv-offloading-backend lmcache

参数定义 (vllm) 定义在 vllm/config/cache.py 中

```
KVOffloadingBackend = Literal["native", "lmcache"]

kv_offloading_backend: KVOffloadingBackend = "native"

"""The backend to use for KV cache offloading. Supported backends include
'native' (vLLM native CPU offloading), 'lmcache'.
KV offloading is only activated when kv_offloading_size is set."""
```

关联参数: --kv-offloading-size

选择 KV Cache 卸载后端。lmcache 后端使用 <https://github.com/LMCache/LMCache> 库，支持将 KV Cache 从 NPU 显存卸载到 CPU 内存，并提供持久化 KV Cache 以及分布式 KV Cache 共享能力。

调用链分析：

```

CLI: --kv-offloading-backend lmcache --kv-offloading-size 8
|
▼
CacheConfig.kv_offloading_backend = "lmcache"      # vllm/config/cache.py:147
|
▼
vllmConfig._post_init_kv_transfer_config()          # vllm/config/vllm.py:622-657
|  ├── kv_connector = "LMCacheConnectorV1"
|  ├── kv_connector_extra_config = {
|  |   ├── "lmcache.local_cpu": True,
|  |   ├── "lmcache.max_local_cpu_size": kv_offloading_size / num_kv_ranks
|  |   └── }
|  └── 设置 LMCACHE_USE_EXPERIMENTAL, LMCACHE_CHUNK_SIZE 等环境变量
|
▼
KVConnectorFactory.create_connector()                # vllm/distributed/kv_transfer/kv_connector/factory.py
|
▼
LMCacheConnectorV1                                  # vllm/distributed/kv_transfer/kv_connector/v1/lmcache_connector.py
|  ├── load_kv_layer() / save_kv_layer()            # 逐层异步加载/保存 KV Cache
|  ├── wait_for_layer_load()                        # 等待指定层的 KV 加载完成
|  └── 底层依赖 lmcache 库进行 CPU 内存管理和数据传输
|
▼
Model Execution (per-layer)                          # 在模型逐层前向传播中嵌入 KV 搬运

```

组件	vllm (已实现)	vllm-ascend
参数定义	CacheConfig.kv_offloading_backend — Literal["native", "lmcache"]	platform.py 中无验证/处理逻辑
配置桥接	post_init_kv_transfer_config()完整实现 lmcache → LMCacheConnectorV1 映射及环境变量配置	未 patch 该方法, lmcache 分支在 Ascend 上未正确映射
Connector 实现	LMCacheConnectorV1 — 逐层异步 load/save KV, layer-wise 事件聚合	lmcache_ascend_connector.py 仅 re-export 上游类, 无任何 NPU 适配
Factory 注册	LMCacheConnectorV1 已注册	LMCacheAscendConnector 已注册
异步传输基础设施	torch.cuda.Stream/Event + CUDA Graph PIECEWISE	CpuNpuOffloadingHandler 已实现 torch.npu.Stream/Event+ torch.ops._C_ascend.swap_blocks
Cli 支持	--kv-offloading-backend lmcache 正常工作	该参数在 Ascend 上无法正常工作
端到端测试	上游 lmcache 测试用例	无任何 lmcache 后端测试
文档	上游文档	仅有手动部署指南, 无自动化配置说明

上游 vLLM 的 lmcache 后端实现完整且成熟，但 vllm-ascend 侧仅有空壳 connector 注册。核心缺口在于：(1)配置桥接——将--kv-offloading-backend lmcache 正确映射到 Ascend connector；(2)NPU 适配——用 CpuNpuOffloadingHandler 的异步传输能力替换 CUDA 依赖。

技术难点：

1. 配置桥接，post_init_kv_transfer_config()中 lmcache 分支需要将 kv_connector 映射为 "LMCacheAscendConnector"而非"LMCacheConnectorV1"，并添加 NPU 特定配置。需通过 vllm-ascend 的 patch 机制覆盖上游逻辑。
2. LMCache 库兼容性，LMCache 库内部可能使用 CUDA 特定 API（如：cudaMalloc、cudaMemcpy），需验证其在 Ascend torch_npu 环境下是否可用。
3. ACL Graph PIECEWISE 兼容，上游要求 CUDA Graph 使用 PIECEWISE 模式以支持逐层 KV 操

- 作。Ascend ACL Graph 是否支持类似的逐层捕获机制需验证。
4. 异步 NPU-CPU 数据搬运, vllm_ascend/kv_offload/cpu_npu.py 中已有 CpuNpuOffloadingHandler 使用 torch.npu.Stream/Event + torch.ops._C_ascend.swap_blocks()的参考实现, 但需整合到 LMCache 框架中。
 5. torch.cuda.is_current_stream_capturing(), vllm-ascend 在 acl_graph.py 中已有 ACL Graph 管理代码, 可能需通过标志位或状态查询提供等价判断。
 6. LMCache 环境变量配置, LMCACHE_LOCAL_CPU, LMCACHE_MAX_LOCAL_CPU_SIZE 等为纯配置项, 设备无关。

2.--mm-encoder-only

参数定义 (vllm) 定义在 vllm/config/multimodal.py 中

```
mm_encoder_only: bool = False
"""
When enabled, skips the language component of the model.

This is usually only valid in disaggregated Encoder process.
"""
```

启用后仅运行多模态编码器 (如 ViT), 跳过语言模型 (LLM Decoder) 部分。典型用于 Disaggregated Encoder 架构——Encoder 进程只负责处理图像/视频/音频特征, 将编码结果传递给 Decoder 进程。

调用链分析:

```
CLI: --mm-encoder-only
|
▼
MultiModalConfig.mm_encoder_only = True          # vllm/config/multimodal.py:137
|
▼
Model Interface (_mark_language_model)           # vllm/model_executor/models/interfaces.py:236
|   └─ 当 mm_encoder_only=True 时, 跳过 LM 权重初始化
|
▼
GPUModelRunner._execute_mm_encoder()              # vllm/v1/worker/gpu_model_runner.py
|   └─ 条件: has_ec_transfer() and is_producer
|   └─ 仅执行 MM Encoder 前向传播
|   └─ 返回 make_empty_encoder_model_runner_output()
|
▼
EncoderOnlyAttentionSpec                          # vllm/v1/kv_cache_interface.py
|   └─ encoder-only 层使用不带 KV Cache 的注意力机制
|
▼
EncoderOnlyAttention                             # vllm/model_executor/layers/attention/encoder_only_attention.py
|   └─ causal=False, 直接使用 Q/K/V 张量, 无需 KV Cache
```

组件	vllm (已实现)	vllm-ascend
参数定义	MultiModalConfig.mm_encoder_only: bool = False	platform.py 未对 mm_encoder_only 做显式验证/兼容性处理
Encoder 调度	GPUModelRunner._execute_mm_encoder() — 条件 has_ec_transfer() and is_producer	继承自 GPUModelRunner, 调用路径已存在

Encoder Attention	MMEncoderAttention — 标准 GPU 实现	AscendMMEncoderAttention 基于 torch_npu.npu_fusion_attention 已实现
310P Attention	/	AscendMMEncoderAttention310 已实现
Encoder-Only Attention Spec	EncoderOnlyAttentionSpec — 不带 KV Cache 的注意力	may_add_encoder_only_layers_to_kv_cache_config() 已在 model_runner_v1 中调用
EC Transfer 集成	has_ec_transfer() + is_producer 完整流程	model_runner_v1.py:1168 有条件检查，但未验证完整流程
KV Cache 配置	encoder-only 层正确配置	may_add_encoder_only_layers_to_kv_cache_config()
端到端测试	上游 encoder-only 测试	test_disaggregated_encoder.py 存在但使用 --ec-transfer-config，不使用--mm-encoder-only，无专用测试
文档	上游文档	无 mm-encoder-only 使用指南和适配说明

--mm-encoder-only 在 vllm-ascend 侧的核心执行路径已基本就绪 (encoder attention、model runner、KV cache 配置均已有实现)，仅需补充显式参数验证、EC Transfer 全流程验证、专用测试和文档。

技术难点：

1. MM Encoder 执行路径，AscendMMEncoderAttention 已实现并使用 torch_npu.npu_fusion_attention，forward 路径完整。
2. KV Cache 配置，may_add_encoder_only_layers_to_kv_cache_config()已在 model_runner_v1 中调用。
3. Disaggregated Encoder 集成需验证与 ec_transfer (Encoder-Connector) 的配合在 Ascend 上是否正常工作。
4. 多模型兼容性验证，需验证不同多模态模型（如：Qwen2.5-VL、LLaVA、InternVL）在 encoder-only 模式下的兼容性。
5. ACL Graph 兼容，encoder-only 模式下的 ACL Graph 捕获需验证——空输出是否影响 Graph 重放。
6. 310P 兼容，AscendMMEncoderAttention310 已实现，但需确认 encoder-only 场景下的正确性。

四、技术方案

1. --kv-offloading-backend lmcache 适配方案

核心目标是让--kv-offloading-backend lmcache --kv-offloading-size N 在 Ascend NPU 上正常工作。采用“配置桥接 + NPU 适配层”的策略，利用 vllm-ascend 已有的 patch 机制和 CpuNpuOffloadingHandler 参考实现。

Step 1: 配置桥接 — Patch_post_init_kv_transfer_config

vllm_ascend/patch/platform/下添加 patch，覆盖上游配置逻辑，将 lmcache 分支中的 connector 映射为 LMCacheAscendConnector：

```
# 目标行为：当 kv_offloading_backend == "lmcache" 时
self.kv_transfer_config.kv_connector = "LMCacheAscendConnector" # 而非 "LMCacheConnectorV1"
self.kv_transfer_config.kv_connector_extra_config = {
    "lmcache.local_cpu": True,
    "lmcache.max_local_cpu_size": kv_gb_per_rank,
    # NPU 特定配置
}
```

Step 2: 充实 LMCacheAscendConnector

- 继承 LMCacheConnectorV1 基本框架
- 替换底层 CUDA 数据搬运为 NPU 等价操作 (torch.npu.Stream/Event)
- 集成 CpuNpuOffloadingHandler 的异步传输模式
- 处理 NPU 特定的 swap_blocks 操作 (torch.ops._C_ascend.swap_blocks)

Step 3: ACL Graph 兼容性

- 验证 LMCache 逐层操作与 ACL Graph 的兼容性
- 参考 vllm_ascend/compilation/acl_graph.py 的实现

Step 4: LMCache 库兼容性验证

- 验证 LMCache 库在 torch_npu 环境下的基本功能
- 检查是否有硬编码的 CUDA 依赖 (cudaMalloc、cudaMemcpy 等)

Step 5: 测试与验证

- 单元测试: 验证 connector 的加载/保存/等待操作
- 集成测试: 使用真实模型验证 KV Cache 卸载功能
- 性能测试: 对比不同 kv_offloading_size 下的延迟和吞吐

涉及文件:

文件	修改类型	修改内容
vllm_ascend/distributed/kv_transfer/kv_pool/lmcache_ascend_connector.py	重写	从空壳包装升级为 NPU 适配实现
vllm_ascend/patch/platform/	新增	Patch post_init_kv_transfer_config()的 lmcache 分支
vllm_ascend/platform.py	修改	添加 kv_offloading_backend 的验证逻辑
vllm_ascend/utils.p	修改	扩展 check_kv_extra_config()支持 lmcache 配置
tests/ut/test_lmcache_ascend_connector.py	新增	单元测试
tests/e2e/test_kv_offloading_lmcache.py	新增	端到端测试

2. --mm-encoder-only 适配方案

由于 AscendMMEncoderAttention 已实现, 且 model_runner 中 encoder 执行路径已存在, 适配主要围绕**验证+补充+测试**展开:

Step 1: 参数验证与平台兼容性声明

在 vllm_ascend/platform.py 的 validate_and_override_config()中添加 mm_encoder_only 的验证逻辑:

```
# platform.py - validate_and_override_config()
multimodal_config = getattr(model_config, "multimodal_config", None)
if multimodal_config and multimodal_config.mm_encoder_only:
    # 验证 Ascend 支持 mm_encoder_only 模式
    logger.info("mm_encoder_only is enabled on Ascend platform.")
    # 检查 ec_transfer 是否正确配置 (可选)
```

Step 2: 验证 Disaggregated Encoder 路径

确认 has_ec_transfer() and get_ec_transfer().is_producer 条件在 Ascend 上正确触发 encoder-only 执行路径。

Step 3: ACL Graph 兼容性验证

验证 encoder-only 模式下的 ACL Graph 行为:

- encoder 前向传播能否被正确捕获
- make_empty_encoder_model_runner_output()返回的空输出是否影响 Graph 重放

Step 4: 端到端测试

使用主流多模态模型 (Qwen2.5-VL、LLaVA 等) 编写测试用例验证完整流程。

涉及文件:

文件	修改类型	修改内容
vllm_ascend/platform.py	修改	添加 mm_encoder_only 参数验证
tests/e2e/singlecard/test_mm_encoder_only.py	新增	端到端测试
docs/source/user_guide/feature_guide/mm_encoder_only.md	新增	使用文档

3. 计划使用的 AI 工具及辅助方式

工具	用途	辅助方式
Claude Code Agent 框架	开发流程自动化	Agent 调度、多步骤任务编排、工具调用 (代码搜索/文件编辑/命令执行)、多 Agent 并行协作
GLM-5	核心推理与生成	代码生成、代码审查、调试分析、torch_npu API 解读、错误日志分析、方案讨论

五、开发时间规划

环境准备与熟悉			
环境搭建	4/16-4/18	搭建 vllm-ascend 开发环境，确认 torch_npu 可用，跑通现有测试	可运行的开发环境
上游代码熟悉	4/21-4/25	深入阅读 vLLM 中 LMCacheConnectorV1 和 mm_encoder_only 的完整实现；阅读 vllm-ascend 中 CpuNpuOffloadingHandler 和 patch 机制	代码阅读笔记
--mm-encoder-only 适配			
参数验证	4/28-4/29	在 platform.py 添加 mm_encoder_only 验证；确认 ec_transfer 路径兼容	代码 patch
集成测试	4/30-5/8	编写端到端测试，使用 Qwen2.5-VL 等模型验证 encoder-only 模式	测试用例
问题修复	5/9-5/15	修复测试中发现的问题 (ACL Graph 兼容性、模型特定问题等)	Bug fix
文档编写	5/16-5/19	编写使用文档和适配说明	文档
审查修改	5/20-5/22	内部 review、修改	
--kv offloading-backend lmcache 适配			
LMCache 库评估	5/26-5/28	安装 lmcache 库，在 torch_npu 环境下评估其	评估报告

		CUDA 依赖程度，确定适配范围	
配置桥接实现	5/29-6/2	Patch _post_init_kv_transfer_config()的 lmcache 分支，使 connector 映射为 LMCacheAscendConnector	配置桥接代码
Connector NPU 适配	6/3-6/9	重写 lmcache_ascend_connector.py，集成 CpuNpuOffloadingHandler 的 NPU Stream/Event 异步传输	核心适配代码
ACL Graph 兼容	6/10-6/11	验证并处理逐层 KV 操作与 ACL Graph 的兼容性问题	兼容性 patch
集成验证	6/12-6/16	使用真实模型验证 KV Cache 卸载/加载的正确性	测试报告
测试完善	6/17-6/18	编写单元测试和端到端测试	测试用例
Bug 修复	6/19-6/26	修复问题	
收尾与文档完善			
最终检查	6/29-6/30	检查两个参数的完整适配质量，确保代码规范和测试通过	检查报告
文档收尾	7/1-7/2	完善使用文档，更新支持矩阵	最终文档
预留缓冲	7/3	应对突发问题	

六、预期成果

1、功能成果

--kv-offloading-backend lmcache

成果	说明
CLI 参数可用	--kv-offloading-backend lmcache、--kv-offloading-size N 在 vllm-ascend 上可正常使用
KV Cache 卸载	成功将 KV Cache 从 NPU 显存卸载到 CPU 内存，减少显存占用
异步传输	利用 NPU Stream/Event 实现异步 KV 数据搬运，隐藏传输延迟
精度一致	卸载前后模型推理精度一致（KV Cache 数据无损失）
测试覆盖	新增 3 个测试用例（connector 单元测试、端到端集成测试、不同模型验证）

--mm-encoder-only

成果	说明
CLI 参数可用	--mm-encoder-only 在 vllm-ascend 上可正常开启，与 Disaggregated Encoder 架构配合工作
模型支持	至少验证 Qwen2.5-VL 系列、LLaVA 系列
精度一致	Ascend 上 encoder-only 模式的输出特征与 GPU 上游一致
测试覆盖	新增 2 个端到端测试用例覆盖典型场景
--mm-encoder-only	CLI 参数可用

2、性能指标

--kv-offloading-backend Imcache

指标	目标
NPU 显存节省	KV Cache 成功卸载到 CPU，显存节省量与 kv_offloading_size 配置一致
异步传输延迟隐藏	NPU Stream 异步传输延迟被计算重叠 $\geq 50\%$
端到端吞吐	Imcache 后端在 Ascend 上的端到端吞吐可接受，无明显性能瓶颈

--mm-encoder-only

指标	目标
Encoder 前向推理延迟	与 GPU encoder-only 模式延迟相当（NPU 算力差异范围内）
特征输出精度	cos similarity > 0.999（与 GPU 参考输出对比）

七、团队介绍

姓名	单位/学校	角色	主要分工	相关经验/技能
乔凡超	华东师范大学	队长	总体适配方案设计、节奏把控	Agent 使用技巧、AI infra 开发、微架构优化